PhotoMatch — Guía Maestra de Defensa Oral

Parte 2: Ciclo Completo de Ingeniería de Software (TP5 a TP9)

Ingeniería de Software III · Universidad Católica de Córdoba (UCC) · Ciclo 2026

Alumno: Lautaro Alejo Vallino
Legajo: 2301009 · Comisión 2
Defensa Oral: Viernes 6/11/2026 (13:30 ART)

Releases Oficiales: `v5.0.0` a `v9.0.0` (100% completas)
Repositorio: github.com/LautyUCC/ingesoft3
Tiempo Asignado: 10 minutos

1. Visión Holística del Pipeline de PhotoMatch

El flujo completo de ingeniería de software implementado garantiza calidad, trazabilidad y seguridad desde el código fuente hasta la operación en vivo:

1. **Commit & Push:** Protegido por Secret Scanning y Push Protection en GitHub.
2. **Pull Request:** Dispara análisis SAST con CodeQL, auditoría SCA con Dependabot y el Quality Gate de cobertura con `c8` (≥80% líneas, ≥70% ramas). El **PR 17** permanece abierto y bloqueado en rojo como evidencia contractual permanente.
3. **Integración en `main`:**
   - `build-backend` y `build-frontend`: Compilan y empaquetan imágenes Docker una sola vez, exportando `.tar`.
   - `publish`: Publica en GHCR con etiqueta inmutable `sha-<COMMIT_SHA>`.
   - `deploy-qa`: Despliega en QA (`http://127.0.0.1:3001`) y corre `smoke.sh`.
   - `integracion`: Ejecuta 3/3 pruebas de API REST con Playwright contra PostgreSQL real.
   - `e2e`: Ejecuta 3/3 flujos completos con Chromium headless. Si falla, **deploy-prod se bloquea y cancela automáticamente** (evidencia real en CI Run 40283556436, PR 37).
   - `deploy-prod`: Requiere aprobación humana de `LautyUCC` en GitHub Environments. Despliega en Producción (`http://127.0.0.1:3002`), supervisado por Sentry y UptimeRobot.

2. Matriz Consolidada de Entregables Oficiales

TP	Eje Temático	Release	Evidencia Principal y Demostración
TP5	Testing y Calidad	`v5.0.0`	Quality Gate con `c8`. **PR 17 abierto y bloqueado** en rojo por cobertura (64,2% < 80%).
TP6	Continuous Delivery	`v6.0.0`	Pipeline desacoplado «Build Once, Deploy Many». **Gate rechazado** (CI 40277582164) y **Rollback real en 7,42 s**.
TP7	Contenedores y E2E	`v7.0.0`	**Gate E2E Rojo Demostrado:** PR 37 / CI 40283556436 (E2E falló, PROD bloqueado). Recuperación verde en PR 39 (CI 40284305415).
TP8	Infraestructura (IaC)	`v8.0.0`	Módulo Terraform `infra/` con 8 recursos. **Ciclo de vida en vivo:** preprod en :3003, **idempotencia estricta** y destroy limpio.
TP9	DevSecOps y Observabilidad	`v9.0.0`	Dependabot PR 30 merged, CodeQL verde, mitigación ReDoS y DoS. **Sentry:** Error real PHOTOMATCH-BACKEND-1 -> **Issue 38** en Projects. **UptimeRobot:** Incidente 503/UP.

3. Guion Estratégico de Exposición (10 Minutos)

Distribución Temporal Estricta

- **Min 0 a 2 (TP5):** Mostrar PR 17 bloqueado en vivo. Enfatizar que la calidad es un requisito contractual bloqueante.
- **Min 2 a 4 (TP6):** Mostrar el flujo «Build Once, Deploy Many», la corrida rechazada justificada y el rollback en 7,42 s.
- **Min 4 a 6 (TP7):** Enseñar la corrida roja de CI 40283556436 donde E2E bloqueó Producción, y la posterior recuperación verde en CI 40284305415. Mostrar curl a :3001 y :3002.
- **Min 6 a 8 (TP8):** Ejecutar `terraform plan` (8 altas), `apply` levantando :3003, segundo `plan` mostrando 0 cambios (idempotencia) y `destroy` limpio.
- **Min 8 a 10 (TP9):** Mostrar PR Dependabot, CodeQL verde, error fatal en consola Sentry, Issue 38 en GitHub Project y monitor UptimeRobot con incidente 503/UP.

4. Preguntas Cruciales para la Mesa Examinadora

Pregunta 1: ¿Cómo demostraron que una falla funcional en la interfaz no llega a producción?

Respuesta para defender:

En el PR 37 rompimos la ruta del formulario HTML de clientes. En la corrida de GitHub Actions 40283556436, la integración pasó 3/3 verde contra la API, pero la suite E2E de Playwright falló en rojo. Dado que `deploy-prod` depende de `needs: e2e`, GitHub Actions canceló y omitió el despliegue a producción automáticamente, manteniendo PROD intacta en el commit sano.

Pregunta 2: ¿Por qué es superior el principio «Build Once, Deploy Many» frente a recompilar en cada ambiente?

Respuesta para defender:

Porque garantiza paridad binaria absoluta. Si se reconstruye la imagen en Producción, una variación en repositorios de npm o paquetes base podría introducir una vulnerabilidad o defecto que no existía cuando se probó en QA. En PhotoMatch, QA y PROD despliegan exactamente el mismo digest inmutable publicado en GHCR.

Pregunta 3: ¿Qué es la idempotencia en Terraform y cómo la auditaron?

Respuesta para defender:

Es la capacidad de ejecutar el aprovisionamiento repetidas veces asegurando que el estado real no sufra mutaciones si ya coincide con la declaración. Tras aplicar la infraestructura de preprod en :3003, ejecutamos un segundo `terraform plan` que devolvió: *«No changes. Your infrastructure matches the configuration»*.

Pregunta 4: ¿De qué manera conectaron la observabilidad técnica con la gestión ágil?

Respuesta para defender:

A partir de la excepción fatal real `terminating connection due to administrator command` capturada en Sentry tras un corte de base de datos, analizamos la causa raíz técnica y creamos la Issue 38 en GitHub, vinculándola al tablero oficial de GitHub Projects en la columna Todo para implementar reconexión automática en el pool.

5. Checklist para Demostración en Vivo frente al Docente

- [ ] **Terminal 1:** Docker containers activos (`photomatch-qa` en :3001 y `photomatch-prod` en :3002).
- [ ] **Terminal 2:** En `infra/`, ejecutar `terraform plan` y `terraform apply` levantando :3003 en vivo.
- [ ] **Navegador Tab 1:** PR 17 abierto y bloqueado en rojo (Quality Gate TP5).
- [ ] **Navegador Tab 2:** CI Run 40283556436 con E2E rojo y `deploy-prod` en gris (Skipped) (TP7).
- [ ] **Navegador Tab 3:** CI Run 40284305415 con recuperación verde y promoción a Producción (TP7).
- [ ] **Navegador Tab 4:** Consola Sentry con incidente PHOTOMATCH-BACKEND-1 y GitHub Project con Issue 38
- [ ] **Navegador Tab 5:** Dashboard de UptimeRobot con incidente 503/UP y releases `v5.0.0` a `v9.0.0`.